Class and Object

Section 9.1 – 9.2

Use Hamburger class and Person class as examples

Define a class

- **Declare Data Members**
  By declaring data members as **private**, you ensure that only the class's own constructors and methods can access or modify them. External classes or functions must interact with this data through **public methods**.
- **Define Constructors**
  Initialize your data members and set up the object's initial state.

Define a class: II

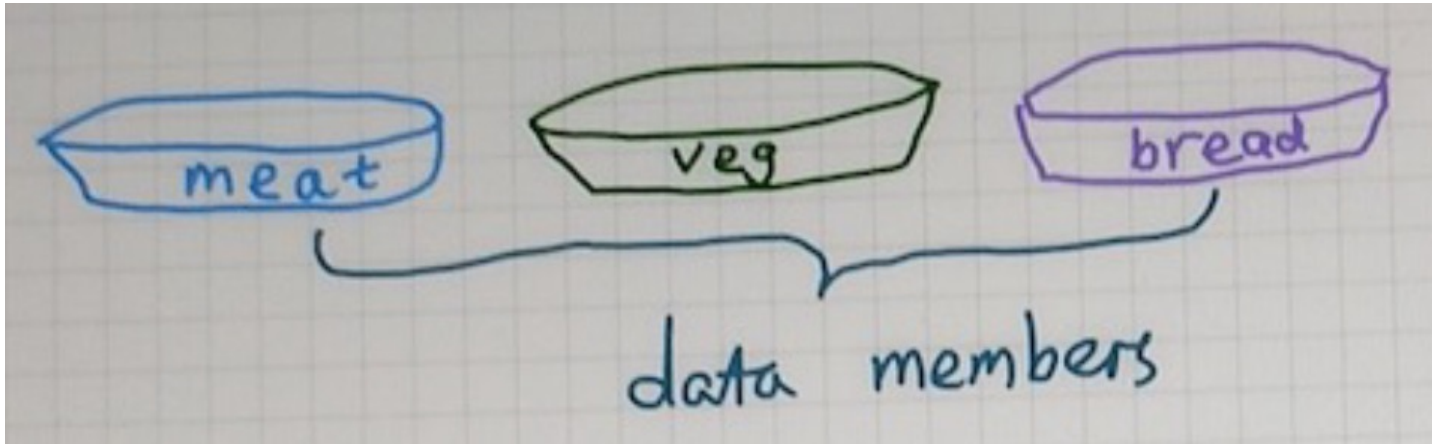
- **Define Methods (Accessors and Modifiers)**
  Create public **getters** (accessors) to retrieve private data and **setters** (modifiers) to update it safely.
- **Define the Destructor**
  Implement a destructor to release resources if any data members utilize **dynamically allocated memory**, preventing memory leaks.

Hamburger class

an object (aka instance) of Hamburger

data members of Hamburger class

- Each hamburger has bread-, vegetable-, and meat- layer.
- data members
  - bread
  - veg (short for vegetable)
  - meat



Constructors of Hamburger

- Purpose of constructor: initialize data members.
- A constructor is like a hamburger maker, who takes requirements – as actual parameters -- to make a hamburger.
- Unlike a human hamburger maker, a constructor of Hamburger class also create operations for those data members.
  - Operation (aka methods): set of instructions

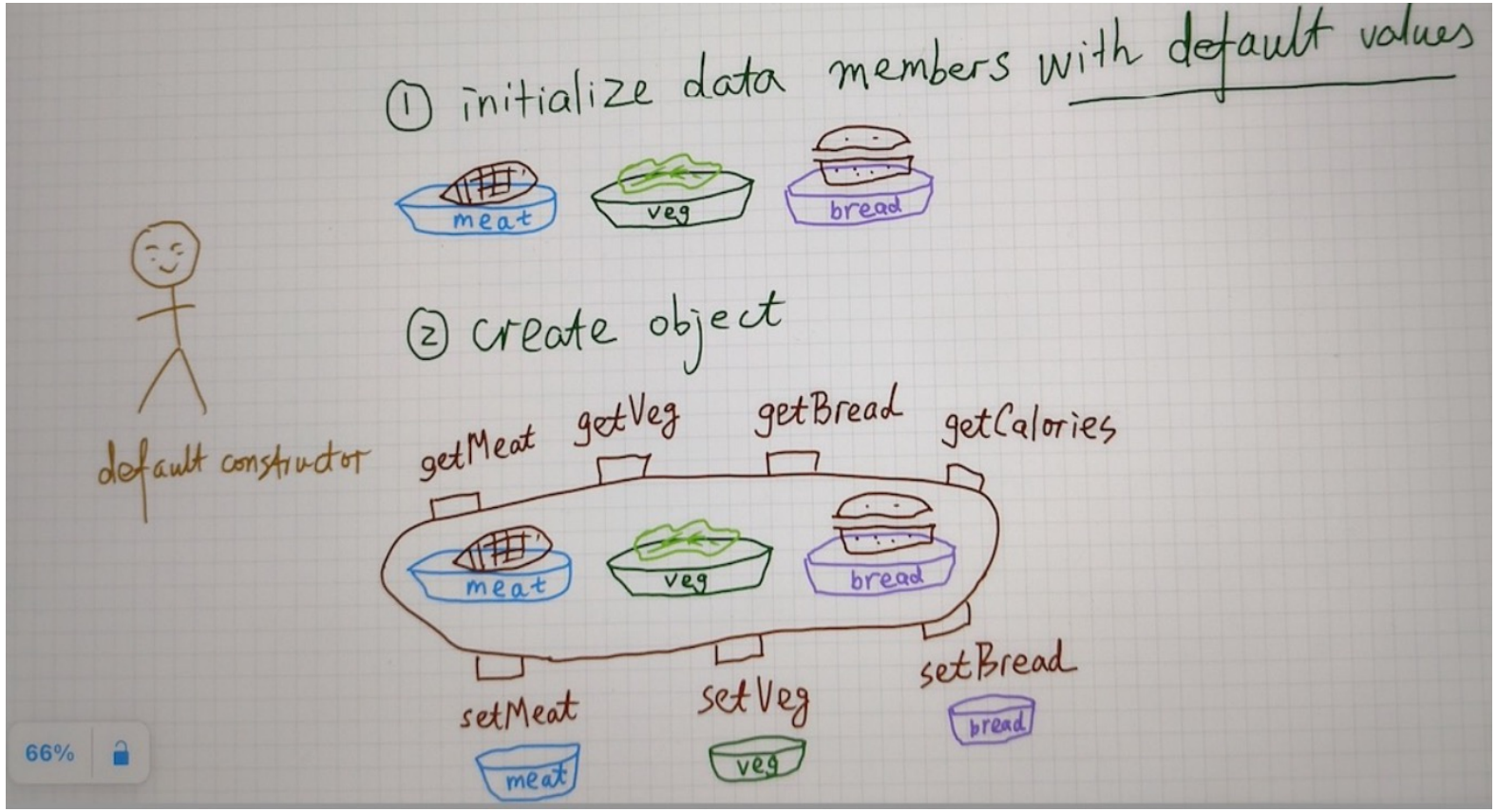
Constructors of Hamburger: II

Can have multiple constructors with different parameters lists

- different number of parameters, for example, one has no parameter, the other has 3 parameters
- Different type of parameters, for example, one takes int, the other takes string
- different order of parameter types, for example, one has parameter list (string, double), the other has parameter list(double, string)

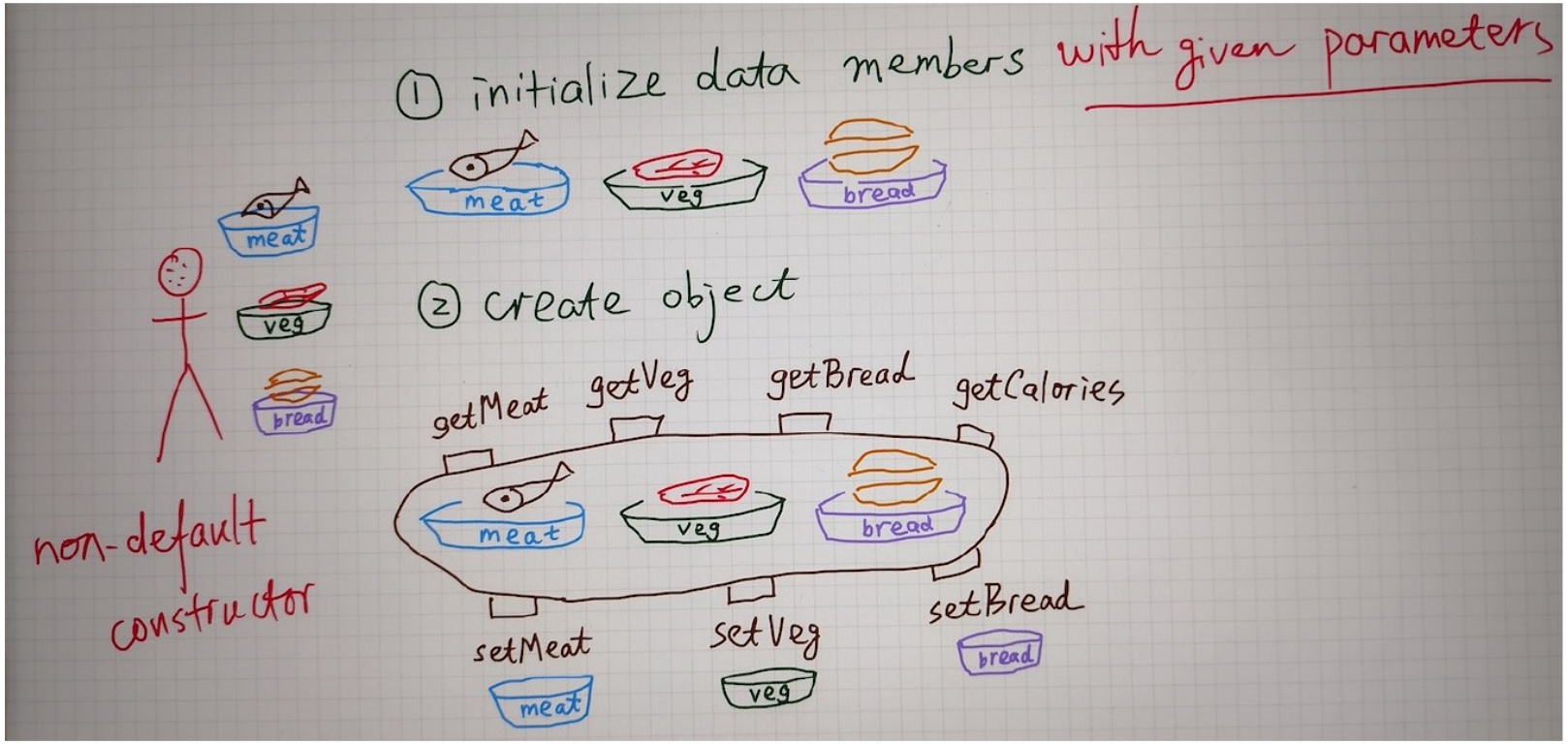
Default Constructor

- Default constructor takes no input and sets data members to default values
- default-configured hamburger (white bread + lettuce + beef).



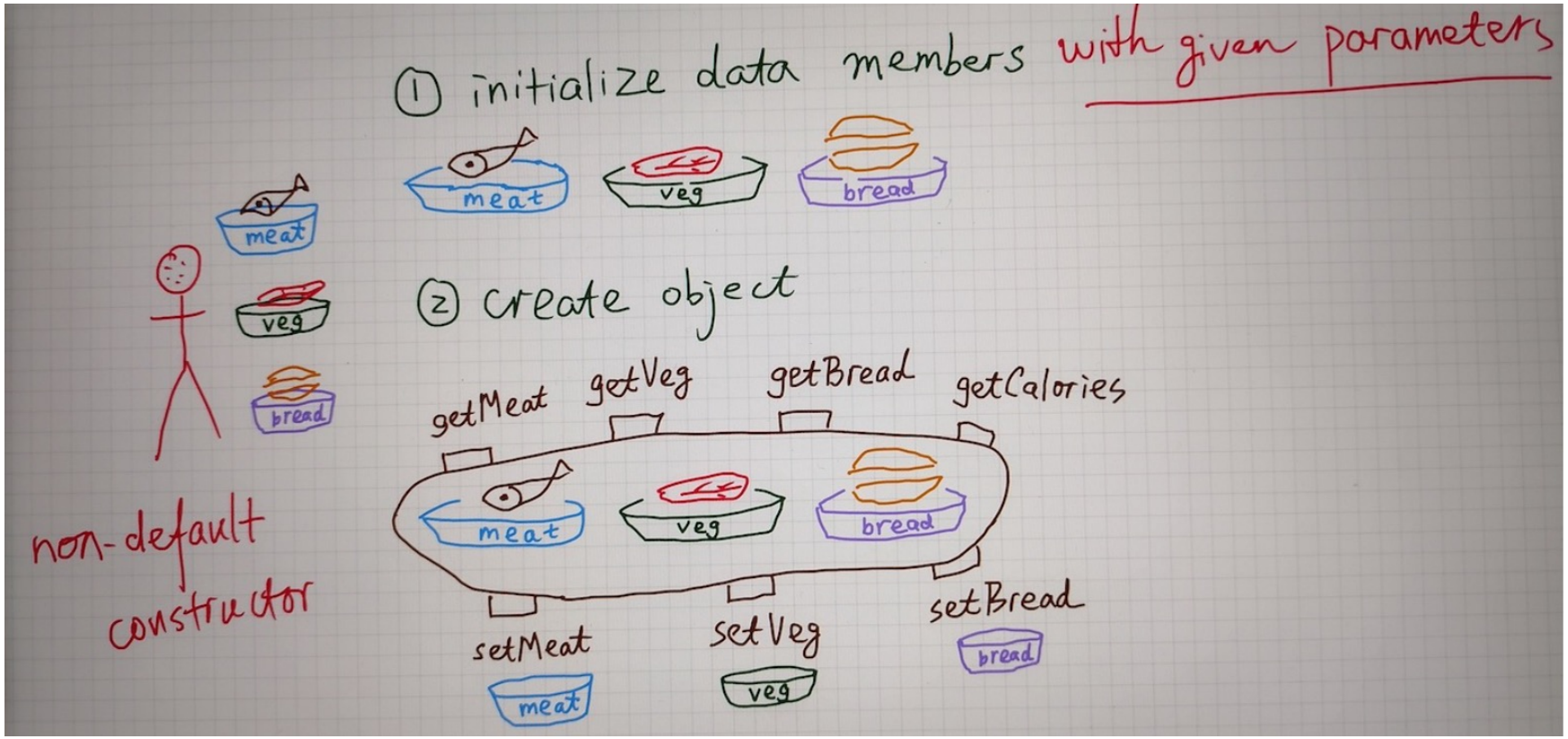
Non-default constructor of Hamburger

- Non-default constructor uses parameters to set data members.
- Make a customer-ordered hamburger, for example, rye bread + onion + chicken



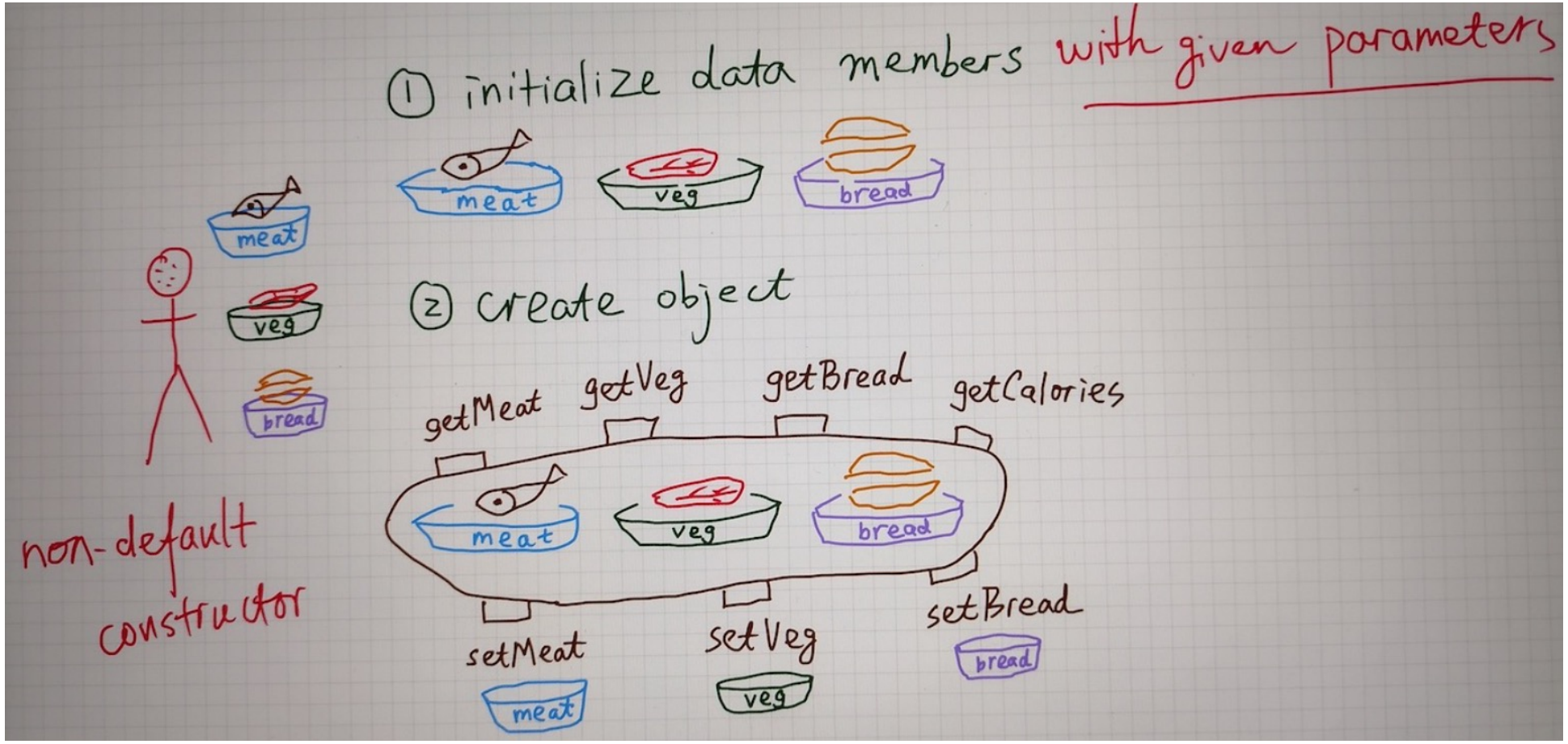
What if a formal parameter has the same name as corresponding data member?

For example, formal parameters of non-default constructor of Hamburger class are called meat, veg, bread, so are the data members.



What if a formal parameter has the same name as corresponding data member? II

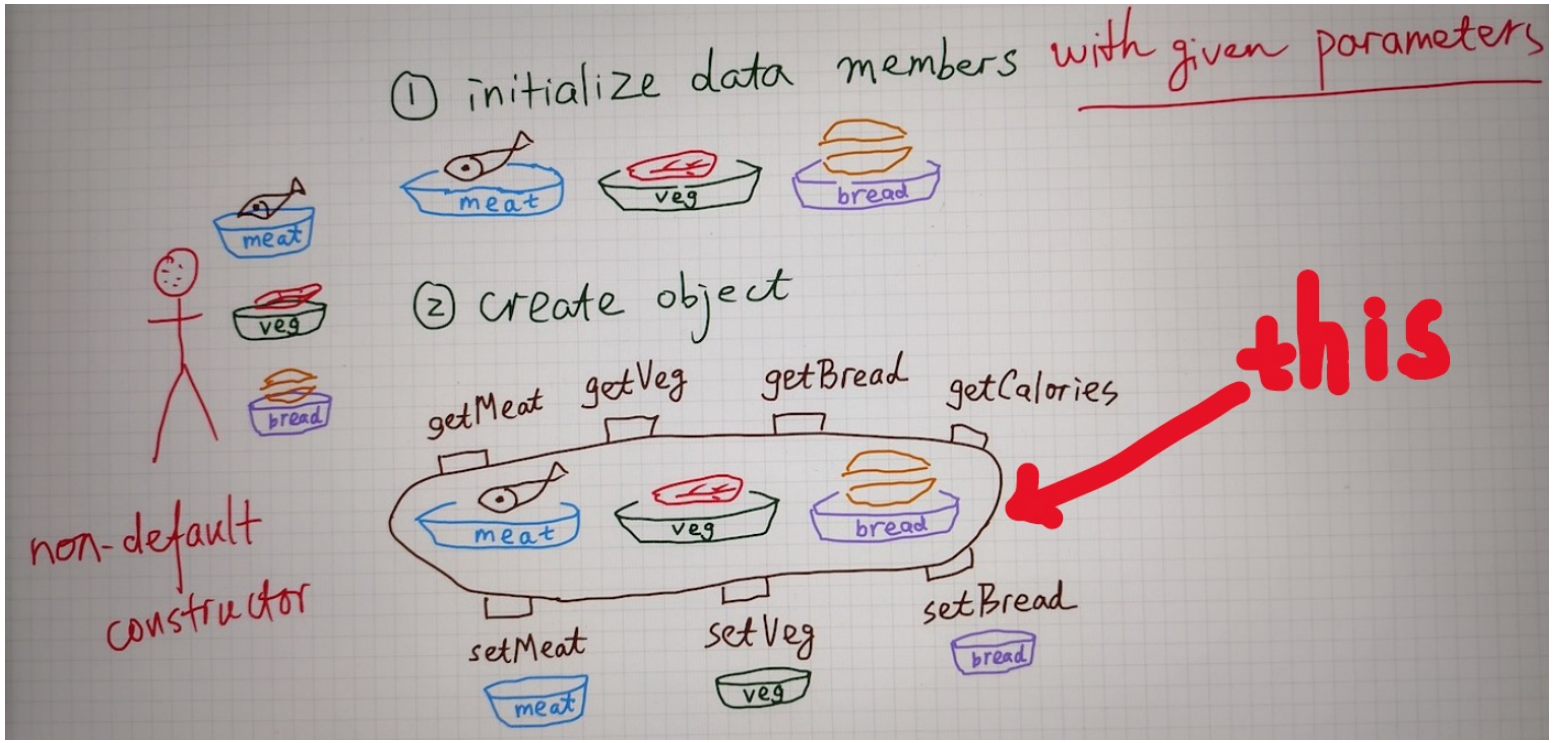
Hamburger::Hamburger(MeatLayer meat, VegLayer veg, BreadLayer bread)



What if a formal parameter has the same name as corresponding data member? III

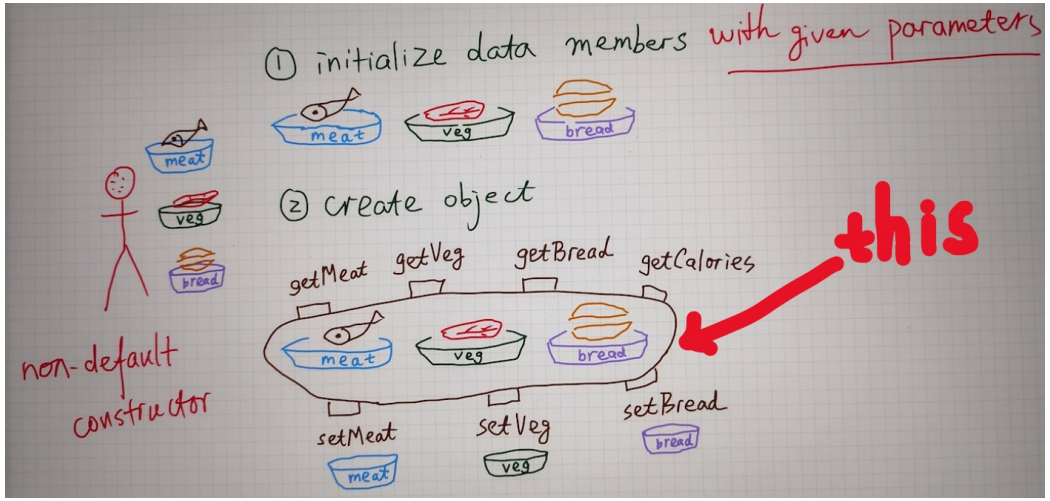
```
Hamburger::Hamburger(MeatLayer meat, VegLayer veg, BreadLayer bread) {
    this->meat = meat;
    this->veg = veg;
    this->bread = bread;
}
```

Keyword this points to the current object.



What if a formal parameter has the same name as corresponding data member? IV

```
Hamburger::Hamburger(MeatLayer meat, VegLayer veg, BreadLayer bread) {
    this->meat = meat;
    //omit rest code ...
}
```



- Keyword this points to current object.
- (*this) is an alias of current object.
- (*this).meat is data member meat of current object.
- (*this).meat can be simplified as this->meat, using arrow operator ->

When no constructor is needed

- If all data members are public, no constructor is needed.
  - User can initialize, modify, or access data directly
- For example, in Lab 9.
  - x, y, z can be any double number.

```
class Coord3D {
public:
    double x;
    double y;
    double z;
};
```

Destructor

- A constructor initialize data members when an object is created.
- The destructor of a class is to wrap up before destroying an object when it is no longer needed.
  - For example, release dynamically allocated memory in constructors. Set the corresponding pointers to be nullptr to avoid dangling pointer problem.
- There is at most one destructor in a class.

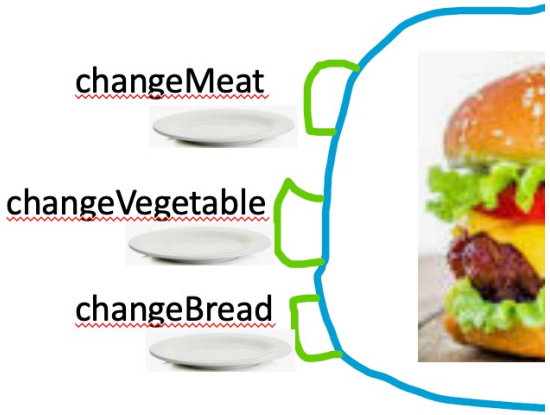
Destructor: II

- Name of the destructor has exactly the same name as class, just add ~ in front of the name.
- no return type, not even void.
- If there is no dynamically allocated memory application in constructor, no need to define the destructor.
  - For example, in FuelTank class, there is only one data member currGasLevel, no dynamically allocated memory. As a result, no need to define the destructor.

Image source: https://www.alamy.com/stock-photo/cartoon-illustration-man-cleaning-up.html

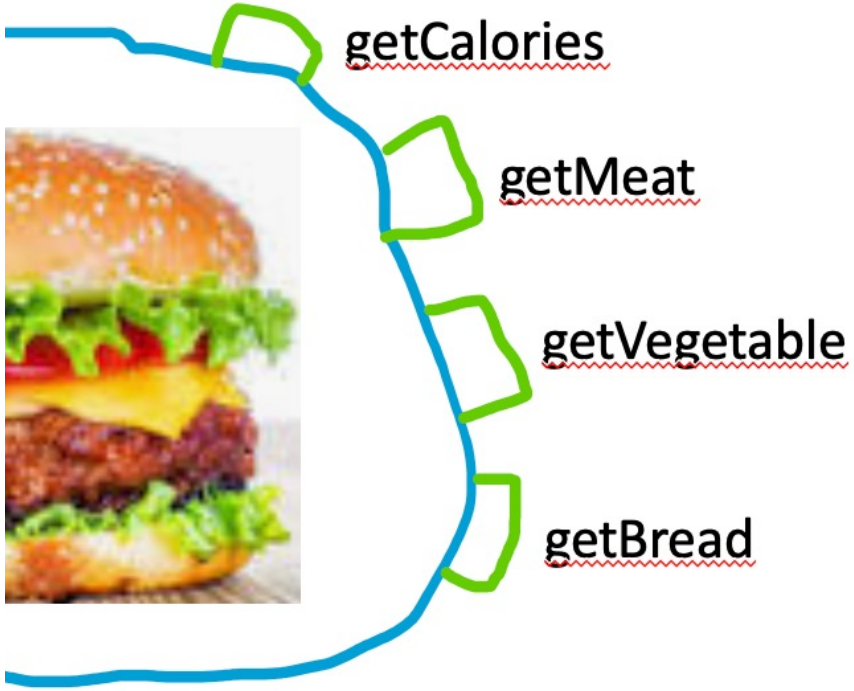
methods of Hamburger: modifier

- Modifiers (setters) change the value of data members
  - change bread layer
  - change meat layer
  - change vegetable layer
- Modifier needs to take parameters.
  - Otherwise, where to hold the new valu
    update the value of current data member?



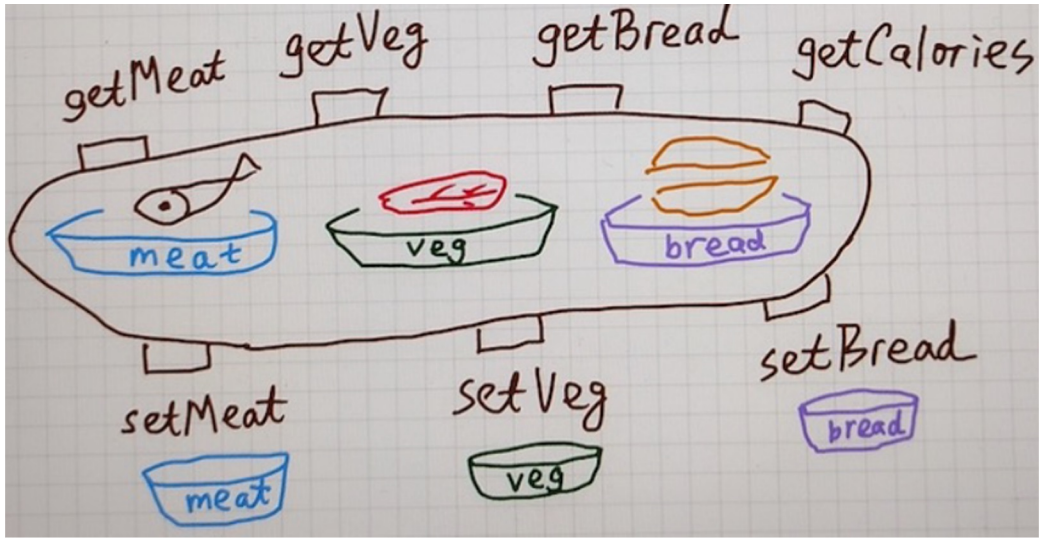
Operations of Hamburger: accessors

- Accessors (getters) get information of data members of the current hamburger
  - Get bread layer
  - Get meat layer
  - Get vegetable layer
  - Calculate calories



For operations (aka methods) of a class

- Data members do not need to be passed as parameters since they can be accessed or modified by operations.
- Think those data members are put in the lounge of the branch (class) and employees in that branch (methods of a class) can access / modify those data members directly.



Person class

- Each person has height.
  - Constructor: initialization, height at birth
  - Modifier (setter): update height at different life stages
  - Accessor (getter): find height at different life stages

image source: https://www.shutterstock.com/image-vector/six-stages-human-life-cycle-birth-1772007914